Automated Software Engineering (2026) 33:70
https://doi.org/10.1007/s10515-026-00605-0

# Hallucination detection in LLM code generation: A sampling-based consensus verification approach

**Taicheng Huang[1] · Zhanhui Ren[1] · Yuan Huang[1] · Xiangping Chen[1] · Yi Liu[2] · Zibin Zheng[1]**

Received: 19 August 2025 / Accepted: 14 February 2026 / Published online: 25 March 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

## Abstract

Large Language Models (LLMs) have revolutionized the code generation task, but their output often contains "hallucinations" - code snippets that look reasonable but are actually wrong (such as API misuse or logic errors). Existing detection methods mainly rely on dynamic code execution, which requires complex runtime environment configurations. This paper proposes HalluCodeDetector, a new static analysis framework based on sampling consistency verification. The method is based on the following assumption: when LLM correctly understands the problem, its random output shows high consistency in syntactic structure, data flow, and API usage patterns. The process of the method is as follows: for a given problem, we let LLM repeatedly generate multiple code samples and evaluate their semantic/functional consistency, a new metric (MRCM) is used to calculate the average similarity between candidate response and other samples to quantify the possibility of hallucination. Experiments on HumanEval+ and MBPP benchmarks demonstrate that HalluCodeDetector achieves AUROC=0.76, outperforming baseline methods like LYNX by 15.2%, and with lower time overhead. Our method provides a secure, efficient, and generalizable solution for improving the reliability of LLM-generated code.

**Keywords** Hallucination detection · Large language model · Code hallucination

## 1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in code generation tasks (Huang et al. 2024), offering solutions that range from simple code snippets to complex software components (Vaithilingam et al. 2022). The advent of AI-powered code generation has fundamentally transformed software development practices (Chen et al. 2025), with almost all developers (92%) using or experiment-

Extended author information available on the last page of the article

ing with AI coding tools (https://github.blog/news-insights/research/the-state-of-open-source-and-ai). These models now routinely assist developers in tasks spanning code generation significantly reducing development cycles and lowering barriers for novice programmers (Wang and Chen 2023).

However, LLMs are prone to generate hallucinations, i.e., code snippets generated by LLMs that appear syntactically correct and even semantically plausible, but may not execute as expected or fulfill specified requirements (Tian et al. 2025). As shown in Fig. 1, the code generated by GPT-4o appears correct but is actually incorrect. The code uses the floor function of the math module, but the math module has not been imported first. While hallucination detection has been explored in natural language contexts (Manakul et al. 2023; Farquhar et al. 2024; Zhang et al. 2024), code generation introduces unique risks: even minor inaccuracies in generated code can propagate into critical software vulnerabilities (Agarwal et al. 2024; Chen et al. 2024; Huang et al. 2018; Huang et al. 2019).

Current approaches for code hallucination detection predominantly rely on dynamic execution (Liu et al. 2023b), which requires pre-configured environments (e.g., specific language interpreters, dependency libraries, and sandboxing tools). This dependency complicates deployment, increases maintenance costs, and introduces compatibility issues across platforms. While recent work like CodeMirage (Agarwal et al. 2024) attempts static analysis through benchmark-driven pattern matching, it focuses on dataset construction rather than providing a generalizable detection methodology. Moreover, recent work has demonstrated that defective commented-out code in the prompt can mislead AI assistants into generating more defective code

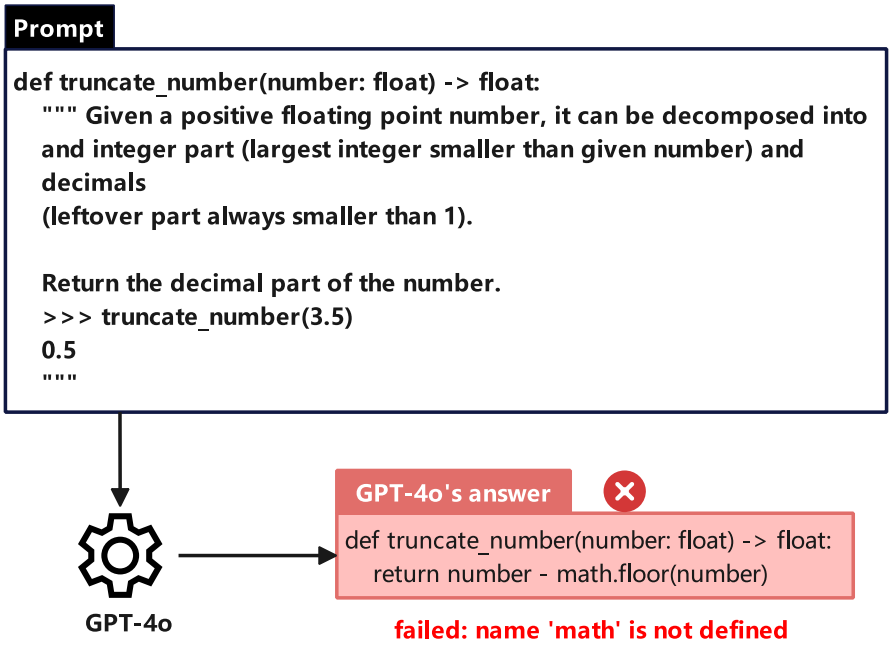



**Fig. 1** An example of code hallucination. The code uses the floor function of the math module, but the math module is not imported first

(Huang el al. 2025), further motivating the need for robust static analysis methods. To the best of our knowledge, there exists no practical method to identify code hallucinations through pure static analysis without requiring runtime environments.

Previous studies (Manakul et al. 2023; Farquhar et al. 2024) in hallucination detection has experimentally validated that LLMs generate semantically consistent outputs for known tasks–rooted in stable internal knowledge–while hallucinated outputs lack such grounding and thus show low consistency across stochastic generations. Complementing this, our pre-experiments further confirm the validity of these insights: on the function-level datasets HumanEval+ and MBPP, the average CodeBLEU consistency score of correct code samples reaches 0.84, whereas that of hallucinated samples is only 0.65. Even on the repository-level dataset EvoCodeBench (Li et al. 2024) which includes complex multi-file tasks, the same trend holds–correct samples achieve an average CodeBLEU consistency score of 0.79, while hallucinated samples drop to 0.53.

Building on these empirical findings, we propose HalluCodeDetector, the first purely static-analysis-based approach for hallucination detection in LLM-generated code. Our sampling-based method operates without requiring external databases, model fine-tuning, or dynamic code execution. The approach is grounded in two key insights derived from the above validations: (1) When an LLM possesses knowledge about a programming problem, its random sampling process will produce responses with high semantic and functional consistency; (2) Conversely, hallucinated outputs exhibit divergence across multiple sampled responses, often manifesting as contradictory logic, erroneous API usage, or syntactically valid but semantically flawed structures.

By computing the average similarity of a candidate response to multiple stochastic generations, HalluCodeDetector quantifies the likelihood of correctness, enabling automated identification of potential hallucinations.

Traditional code similarity metrics like BLEU (Papineni et al. 2002) and CodeBLEU (Ren et al. 2020) prove inadequate for assessing functional consistency in this context. BLEU's lexical matching approach fails to account for code structure equivalence, while CodeBLEU's rigid combination of syntactic and semantic features lacks adaptability to code variations like variable renaming or logic restructuring. Neither metric effectively captures the multidimensional nature of code correctness, particularly regarding API usage patterns and data flow dependencies.

To address these limitations, we propose MRCM (Multi-Response Consistency Metric) – a novel code similarity measure that combines syntactic, semantic, and functional analysis through multi-source feature fusion and dynamic weight allocation. Unlike conventional metrics, MRCM integrates four complementary dimensions: abstract syntax tree (AST) similarity for syntactic structure, data flow similarity for logical dependencies, Levenshtein ratio (Yujian and Bo 2007) for textual edit distance, and Jaccard similarity (Bag and Kumar 2019) for lexical overlap. The metric's adaptive weighting mechanism automatically prioritizes stable discriminative features by analyzing score variances across multiple code samples. We apply MRCM to the code similarity measurement process of HalluCodeDetector.

To evaluate HalluCodeDetector, we used three public datasets, HumanEval+ (Liu et al. 2023a), MBPP (Austin et al. 2021) and EvoCodeBench (Li et al. 2024),

which contain multiple Python code generation tasks. In this paper, hallucinations are judged based on whether they pass all test cases. We conducted experiments with LLM, focusing on hallucination detection of LLM-generated code, and compared with LYNX (Ravi et al. 2024), the SOTA method in the field of LLM hallucination detection. Experimental results show that our method, HalluCodeDetector, has an AUROC score of 0.76, which is 15.2% higher than LYNX. At the same time, our method has much lower time overhead than LYNX, with an average time consumption of 11.9 seconds for LYNX and only 0.016 seconds for HalluCodeDetector.

In summary, this paper makes three primary contributions:

1. We formalize the problem of hallucination detection in code generation as a consistency evaluation task, eliminating the reliance on external databases and pioneering static detection without code execution.
2. We design MRCM, a multidimensional code similarity metric that overcomes the limitations of conventional measures through dynamic feature weighting and joint analysis of syntactic, semantic, and functional code properties.
3. Through extensive experiments on benchmark code generation datasets, we demonstrate that our method achieves significant improvements in hallucination detection accuracy compared to baseline approaches.

To facilitate the research and application, our code and data are available at https://github.com/HalluCodeDetector/HalluCodeDetector.

The remainder of this paper is organized as follows: Section 2 introduces the details of our approach. Section 3 describes the experimental setup. Section 4 reports the experimental results. Section 5 analyzes a case study. Section 6 presents the threats to the validity of our study. Section 7 discusses the related work. We draw conclusions in Section 8.

## 2 Approach

### 2.1 Overview

Figure 2 illustrates the overall framework of HalluCodeDetector. HalluCodeDetector takes some samples generated by the LLM as inputs. For sample i, HalluCodeDetector calculates the average consistency score between the sample and other samples as the HalluCodeDetector score. The lower the HalluCodeDetector score of a sample, the higher the probability that the sample is a hallucination.

**Notation** HalluCodeDetector obtains $N$ stochastic LLM response samples $\{S1, S2, ..., SN\}$ using the same query. The HalluCodeDetector score of the i-th sample, $Score(\mathrm{i})$, such that $Score(\mathrm{i}) \in [0.0, 1.0]$, where $Score(\mathrm{i}) \rightarrow 1.0$ if the i-th sample is true and $Score(\mathrm{i}) \rightarrow 0.0$ if the i-th sample is hallucinated. The following subsection describe the metric MRCM used by HalluCodeDetector to measure code consistency and the specific details of the HalluCodeDetector method.

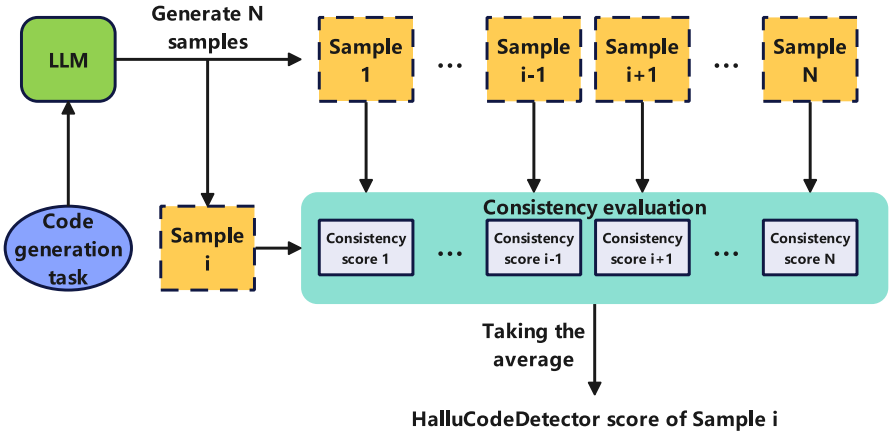



**Fig. 2** The overview of HalluCodeDetector

## 2.2 HalluCodeDetector

Existing code similarity evaluation metrics, such as BLEU and CodeBLEU, struggle to comprehensively capture the syntactic, logical, and semantic consistency of code due to their over-reliance on textual matching or static structural analysis. To address these limitations of conventional methods, this paper proposes a multidimensional dynamic-weight code similarity metric, MRCM (Multi-Response Consistency Metric), which integrates multi-source feature fusion and adaptive weight allocation. MRCM combines AST similarity (capturing syntactic structure), data flow similarity (analyzing logical dependencies), Levenshtein ratio (Yujian and Bo 2007) (measuring textual edit distance), and Jaccard similarity (Bag and Kumar 2019) (evaluating lexical overlap), while automatically adjusting feature weights via a dynamic weighting mechanism. To ensure the reproducibility and efficiency of AST parsing and data flow analysis, we leverage the open-source Python library tree-sitter (Latif et al. 2023).

AST Similarity capture the logical consistency at the grammatical level by comparing the abstract syntax tree structure of two pieces of code (Wen et al. 2019). Assuming that the node sets of the ASTs of the two code samples S1 and S2 are T1 and T2 respectively.

We define MatchedNodes($T_1, T_2$) as the set of mutually exclusive node pairs ($n_1 \in T_1, n_2 \in T_2$), that satisfy structural and syntactic consistency after AST normalization. Before node matching, we normalize both ASTs to eliminate irrelevant noise, including identifier normalization (replace all user-defined identifiers with language-agnostic placeholders), node pruning (remove non-structural nodes that do not contribute to grammatical structure) and subtree standardization (for commutative operations, sort child nodes by their normalized type to avoid false mismatches caused by order invariance). A node pair ($n_1 \in T_1, n_2 \in T_2$) is considered a valid match if and only if all three conditions are satisfied: 1.The syntactic type of $n_1$ equals that of $n_2$; 2.The subtrees of $n_1$ and $n_2$ are consistent; 3.No node in $T_1$ or $T_2$ is included in more than one matched pair.

Their similarity is defined as:

$$\text{AST-Sim}(T_1, T_2) = \frac{2 \cdot |\text{MatchedNodes}(T_1, T_2)|}{|T_1| + |T_2|} \tag{1}$$

Data Flow Similarity analyzes the dependencies of variable definition-use chains based on data flow graphs (DFGs) to identify deep consistency at the logical level (such as the order of variable operations). Assuming that the DFGs of two code samples S1 and S2 are G1=(V1,E1) and G2=(V2,E2) respectively, the similarity is measured by the edge overlap rate:

$$\text{DF-Sim}(G_1, G_2) = \frac{|E_1 \cap E_2|}{|E_1 \cup E_2|} \tag{2}$$

The edit distance is the minimum number of insertion, deletion, and replacement operations. The two code texts are S1 and S2, the Levenshtein rate is calculated as follows:

$$\text{Lev-Ratio}(S_1, S_2) = 1 - \frac{\text{EditDistance}(S_1, S_2)}{\max(|S_1|, |S_2|)} \tag{3}$$

Jaccard Similarity evaluates the lexical overlap of key identifiers, filter out irrelevant symbols:

$$\text{Jaccard-Sim}(S_1, S_2) = \frac{|\text{Tokens}(S_1) \cap \text{Tokens}(S_2)|}{|\text{Tokens}(S_1) \cup \text{Tokens}(S_2)|} \tag{4}$$

MRCM accepts multiple code samples. For each code sample, it calculates the average similarity of each feature against corresponding features in other samples, then calculates the variance of each feature's scores across all samples. Features with higher variance are assigned lower weights, as the variance reflects the stability of a feature across samples. For example, if a feature (e.g., AST similarity) exhibits high variance, it indicates significant fluctuations in its similarity scores across different samples, likely due to oversensitivity to specific code variations, resulting in weak anti-interference capability in its discriminative outcomes. This method adopts the "low-variance, high-weight" principle to adaptively screen features that exhibit stable discriminative power and strong noise resistance, thereby improving the metric's generalization capability in diverse code variation scenarios.

For each sample Si, calculate the variance of its similarity score with all other samples on the four features:

$$\mu_{k,i} = \frac{1}{N-1} \sum_{j=1, j \neq i}^{N} f_{k,i,j} \tag{5}$$

$$\sigma_{k,i}^2 = \frac{1}{N-1} \sum_{j=1, j \neq i}^{N} (f_{k,i,j} - \mu_{k,i})^2 \tag{6}$$

Where k represents the feature (such as AST similarity), $f_{k,i,j}$ represents the similarity score between the i-th sample and the j-th sample on feature k, and $\mu_{k,i}$ represents the average similarity score between the i-th sample and other samples on feature k. $\sigma^2_{k,i}$ represents the variance of the similarity score between the i-th sample and other samples on feature k.

The weight is inversely proportional to the variance, ensuring that features with high stability dominate the decision:

$$\alpha = \frac{1/\sigma^2_{\text{AST}}}{\sum_k (1/\sigma^2_k)} \tag{7}$$

$$\beta = \frac{1/\sigma^2_{\text{DF}}}{\sum_k (1/\sigma^2_k)} \tag{8}$$

$$\gamma = \frac{1/\sigma^2_{\text{Lev}}}{\sum_k (1/\sigma^2_k)} \tag{9}$$

$$\delta = \frac{1/\sigma^2_{\text{Jaccard}}}{\sum_k (1/\sigma^2_k)} \tag{10}$$

Where $\alpha$ represents the weight of AST similarity in MRCM, $\beta$ represents the weight of data flow similarity, $\gamma$ represents the weight of levenshtein rate, and $\delta$ represents the weight of jaccard similarity.

Let *MRCM*((*Si*, *Sn*) denote the MRCM score between samples Si and Sn. HalluCodeDetector with MRCM finds the average MRCM Score of the i-th sample with other samples as the HalluCodeDetector score:

$$\begin{aligned} \text{MRCM}(\text{Si}, \text{Sn}) = & \alpha * \text{AST-Sim}(T_i, T_n) + \\ & \beta * \text{DF-Sim}(G_i, G_n) + \\ & \gamma * \text{Lev-Ratio}(S_i, S_n) + \\ & \delta * \text{Jaccard-Sim}(S_i, S_n) \end{aligned} \tag{11}$$

$$\text{Score}(i) = \frac{1}{N-1} \sum_{n=1, n \neq i}^{N} (\text{MRCM}(\text{Si}, \text{Sn})) \tag{12}$$

To further clarify the calculation process of MRCM and its four core features, we take the example in Fig. 3 (the evaluated sample $S_0$ and four other samples) for step-by-step illustration. We calculate the MRCM score of each sample by following the formulas in Section 2.2, and verify how MRCM captures consistency differences between correct/incorrect samples. The calculation steps are as follows:

Step 1: Calculate Four Core Feature Similarities. We first compute the similarity of each feature between $S_0$ and four other samples. The results are shown in Table 1.

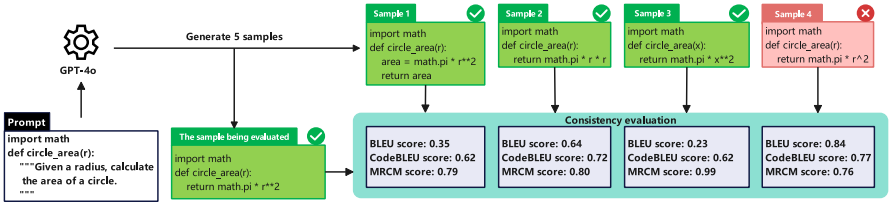



**Fig. 3** GPT-4o generated 4 correct answer samples and 1 incorrect answer sample for a question, and the HalluCodeDetector variant using the MRCM metric gave the most appropriate similarity scores between the evaluated sample and the other 4 samples

**Table 1** The similarity of each feature between $S_0$ and four other samples

| Sample | AST-Sim | DF-Sim | Lev-Ratio | Jaccard-Sim |
|---|---|---|---|---|
| $S_1$ | 0.80 | 0.80 | 0.62 | 0.90 |
| $S_2$ | 0.80 | 0.60 | 0.96 | 0.80 |
| $S_3$ | 1.00 | 1.00 | 0.96 | 1.00 |
| $S_4$ | 0.80 | 0.60 | 0.97 | 0.60 |

**Table 2** The mean, variance, and weights of each feature between $S_0$ and four other samples

| | AST-Sim | DF-Sim | Lev-Ratio | Jaccard-Sim |
|---|---|---|---|---|
| MEAN | 0.85 | 0.75 | 0.88 | 0.83 |
| VARIANCE | 0.007 | 0.028 | 0.022 | 0.021 |
| WEIGHT | 0.51 | 0.14 | 0.17 | 0.18 |

Step 2: Calculate Feature Mean, Variance, and Weights. The results are shown in Table 2.

Step 3: This step directly leverages the feature similarity values in Table 1 and weights in Table 2, substituting them into the MRCM formula (Formula 11) to calculate pairwise consistency scores, then using Formula (12) to compute the average score (HalluCodeDetector score) for $S_0$. As shown in Fig. 3, there is a logical reconstruction between the evaluated sample and sample 1, sample 2. In this case, BLEU and CodeBLEU cannot identify the similarities between samples well, while MRCM performs better in the scenario of logical reconstruction and gives a higher consistency score. A variable is renamed between sample 3 and the sample being evaluated, BLEU and CodeBLEU give very low scores for the consistency of the two samples in this case, while MRCM is almost unaffected by variable renaming. Sample 4 has a logical error, in Python, ^ is a bitwise XOR operator, not a power operator. BLEU and CodeBLEU did not find this logical difference, and gave a relatively higher consistency score, while MRCM gave a more reasonable score.

## 3 Experimental setup

This section describes the experimental setup from three dimensions: dataset, baseline methods, and evaluation metrics. The datasets include function-level (HumanEval+, MBPP) and repository-level (EvoCodeBench) tasks to cover different complexity scenarios. The baseline methods include traditional similarity metric variants, LLM-based methods, and the SOTA model LYNX. The evaluation metrics adopt AUROC

(overall discrimination ability), AUPRC (imbalanced class scenarios), and Spearman's rank correlation coefficient (analysis of sample size impact), ensuring comprehensive evaluation of the proposed method.

## 3.1 Benchmark dataset

### 3.1.1 HumanEval+

HumanEval+ (Liu et al. 2023a) is an advanced benchmarking dataset designed to rigorously evaluate the functional correctness of code generated by LLMs. It extends the original HumanEval+ benchmark (Chen 2021) by addressing critical limitations in test coverage and robustness.

### 3.1.2 MBPP

The MBPP dataset (Austin et al. 2021) is a large-scale evaluation framework designed to assess the code-generation capabilities of AI models, containing 974 Python programming tasks. Introduced by researchers at Microsoft and collaborators, it addresses some limitations of earlier benchmarks like HumanEval+ by scaling up the diversity and complexity of problems while emphasizing real-world practicality.

### 3.1.3 EvoCodeBench

EvoCodeBench (Li et al. 2024) is an evolving code generation benchmark aligned with real-world code repositories. It addresses the poor alignment of existing benchmarks with practical development scenarios by focusing on repository-level code generation tasks. EvoCodeBench includes 275 samples collected from 25 high-quality open-source Python repositories. Each sample is annotated with comprehensive information, including natural language requirements, reference code, explicit dependency paths, and executable test cases.

Each problem in the three datasets contains a number of test cases. If the code generated by the model cannot pass all the test cases, the code is considered to be a hallucination. In the experiment, we use DeepSeek-V3 (https://github.com/deepseek-ai/DeepSeek-V3) to generate 10 answers for each question (calling the model through the API, with the temperature set to 1). The choice of 10 samples per task is motivated by both methodological rigor and practical constraints. Specifically:

1. Representativeness Requirement: Consistency-based detection relies on capturing the variability of LLM outputs for the same task–too few samples fail to cover potential valid output patterns or hallucination variants, leading to biased consistency estimates that cannot reflect the model's true understanding of the task. For example, a model might generate a correct answer once and hallucinate once in 2 samples, but this small sample size cannot distinguish between occasional errors and systematic hallucination.
2. Cost-Efficiency Balance: Excessively large sample sizes would drastically increase computational overhead and experimental cost, significantly extending processing time.

**Table 3** The performance of DeepSeek-V3 in three datasets

| Dataset | Hallucinated | Non hallucinated | Total | Hallucination Ratio |
|---|---|---|---|---|
| HumanEval+ | 458 | 1,182 | 1,640 | 27.9% |
| MBPP | 1,979 | 7,761 | 9,740 | 20.3% |
| EvoCodeBench | 2008 | 742 | 2750 | 73.0% |

### 3.2 Baselines

In order to prove that MRCM is more suitable for HalluCodeDetector than traditional similarity evaluation indicators, we use HalluCodeDetector which uses METEOR (Banerjee and Lavie 2005), BLEU and CodeBLEU as code consistency evaluation indicators as baselines. Since LLM has shown strong capabilities in the field of code understanding, we also use HalluCodeDetector using LLM to evaluate code consistency as a baseline. Furthermore, we include two additional baseline methods representing common practices in hallucination detection: (1) directly querying a powerful LLM as a hallucination detector, reflecting typical developer usage; and (2) LYNX (Ravi et al. 2024), a SOTA open-source model specifically fine-tuned for hallucination detection.

#### 3.2.1 HalluCodeDetector with traditional similarity metrics

The three traditional similarity metrics share the same core logic for HalluCodeDetector: calculate the average similarity between the target sample and all other stochastic generated samples, and use this average as the HalluCodeDetector score for the target sample. The generalized formulation is as follows:

$$\text{Score}_{\text{Metric}}(i) = \frac{1}{N-1} \sum_{n=1, n \neq i}^{N} (\text{Metric}(\text{Si}, \text{Sn})) \tag{13}$$

Where *Metric* denotes the specific similarity metric (METEOR, BLEU, or CodeBLEU), and *Metric*(*Si*, *Sn*) represents the similarity score of sample *Si* relative to sample *Sn* computed by the corresponding metric. The differences between the three metrics are specialized as follows:

**METEOR** Liu et al. (2023a): An automatic text evaluation metric that uses synonym/stem matching and explicit recall mechanisms, and penalizes fragmented outputs to better align with human assessments than basic metrics.

**BLEU** Papineni et al. (2002): A metric originally designed for machine translation quality evaluation, which focuses on surface-level textual matching. Its scores range from 0 to 1, but it does not directly assess grammar or semantic consistency.

**CodeBLEU** Ren et al. (2020): A metric tailored for code generation quality assessment, specifically designed to measure the similarity between code snippets by integrating structural and semantic features of code.

**Fig. 4** Prompt for judging consistency

```
Sample1:
Sample2:
Are the samples functionally consistent?
Just answer "yes" or "no":
```

#### 3.2.2 HalluCodeDetector with prompt

Compared to static similarity metrics, LLMs exhibit better semantic understanding capabilities, enabling them to evaluate the functional consistency between two code samples and identify whether they are consistent or contradictory (Nam et al. 2024). Thus, we query the LLM using the prompts in Fig. 4 to assess whether two samples are functionally consistent. In our experiments, we use GPT-4o (https://openai.com/index/hello-gpt-4o) to evaluate code consistency (model temperature is set to 1).

Let $x_{ij}$ denote whether the i-th sample and the j-th sample are functionally consistent. $x_{ij}$ is 1 if the answer of the LLM is yes, is 0 if the answer of the LLM is no and is 0.5 if the answer of the LLM is other. Then finds the average Score of the i-th sample as the HalluCodeDetector score:

$$Score_{\text{prompt}}(i) = \frac{1}{N-1} \sum_{j=1, j \neq i}^{N} x_{ij} \tag{14}$$

#### 3.2.3 Ordinary LLM

As a foundational baseline, we evaluate the hallucination detection capability of standard large language models without specialized adaptation. This approach leverages the model's inherent confidence estimation. The baseline reflects standard practice where developers directly interrogate LLMs about code correctness. We use GPT-4o for hallucination detection by calling the API with temperature set to 1.

#### 3.2.4 LYNX

LYNX (Ravi et al. 2024) is an open-source LLM specifically designed for detecting hallucinations. Built by fine-tuning Llama-3-70B-Instruct (https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct), Lynx evaluates whether generated answers are faithful to provided contexts without requiring external references. It addresses challenging real-world scenarios through advanced reasoning capabilities, outperforming models like GPT-4o and Claude-3-Sonnet on hallucination detection tasks.

As shown in Fig. 5, for Ordinary LLM and LYNX, we use the same prompt to obtain the answers of LLMs.

```
Given the following QUESTION, TEST and ANSWER you must analyze the provided answer and
determine whether it passes the TEST. Output your final verdict by strictly following this format:
"PASS" if the answer passes the TEST and "FAIL" if the answer does not pass the TEST. Show your
reasoning.
--QUESTION:{question}
--TEST:{test}
--ANSWER:{answer}
--Your output should be in JSON FORMAT with the key "SCORE":{{"SCORE": <your final score>}}
```

**Fig. 5** Prompt for Ordinary LLM and LYNX

### 3.3 Metrics

#### 3.3.1 AUROC

The AUROC (Area Under the Receiver Operating Characteristic Curve), also called AUC-ROC, is a performance metric widely used to evaluate the quality of binary classification models. It quantifies a model's ability to distinguish between positive and negative classes across all possible classification thresholds.

#### 3.3.2 AUPRC

The AUPRC (McDermott et al. 2024) (Area Under the Precision-Recall Curve), also called AUC-PR, is a performance metric used to evaluate binary classification models, particularly in scenarios with class imbalance (e.g., rare positive instances). It measures the trade-off between precision and recall across all classification thresholds, offering a nuanced view of model performance in challenging settings.

#### 3.3.3 Spearman's rank correlation coefficient

Spearman's Rank Correlation Coefficient (Sedgwick 2014) is a non-parametric measure of statistical dependence between two variables. It assesses how well the relationship between the variables can be described by a monotonic function (whether increasing or decreasing).

## 4 Experimental results

### 4.1 Research questions

- **RQ1:(*Effectiveness of* HalluCodeDetector)** Can HalluCodeDetector effectively detect code hallucinations?
- **RQ2:(*Effectiveness of each feature of MRCM*)** To what extent do the four features contribute to MRCM's assessment of similarity?

- **RQ3:(*Efficiency of* HalluCodeDetector)** What is the efficiency of HalluCodeDetector?
- **RQ4:(*The Choice of LLM for samples generation*)** Is HalluCodeDetector effective on different LLMs?
- **RQ5:(*The Impact of the Number of Samples*)** To what extent do number of samples contribute to correctly detect code hallucinations?

The five Research Questions proposed in this paper aim to systematically evaluate the performance, efficiency and generalization ability of the HalluCodeDetector method. RQ1 (validity verification): By comparing indicators such as AUPRC/AUROC, verify whether the HalluCodeDetector based on static consistency verification can surpass existing methods. RQ2 (feature contribution analysis): By conducting an ablation study on the four features (AST, data flow, Levenshtein ratio, and Jaccard similarity) in MRCM, we aim to quantify the contribution of each feature to the overall performance and identify the most critical components for detecting code hallucinations. RQ3 (efficiency analysis): Explore the computational overhead of the method, focusing on verifying the time cost advantage of static analysis over the baseline methods that rely on LLM interaction. RQ4 (cross-model generalization): By testing samples generated by LLMs of different architectures such as GPT-4o and Qwen3, evaluate whether the method is limited to the output style or training paradigm of a specific model, and verify its robustness as a general detection framework. RQ5 (influence of sample size): Analyze the relationship between the number of samples and detection performance, and determine the optimal sampling size to balance computational cost and accuracy.

### 4.2 RQ1: Effectiveness of HalluCodeDetector

**Approach** We adopt AUROC and AUPRC as primary metrics (formally defined in Section 3.3), chosen for their adaptability to hallucination detection:

– AUROC measures the model's ability to distinguish hallucinated from non-hallucinated code across all possible decision thresholds, providing a holistic assessment of general detection performance.
– AUPRC is particularly critical for code hallucination tasks, where datasets often exhibit class imbalance (e.g., 20.3% hallucination rate in MBPP, 73.0% in EvoCodeBench). It focuses on the precision-recall trade-off, ensuring we capture rare hallucinated samples without excessive false positives–essential for practical software development scenarios.

To evaluate the performance of HalluCodeDetector with MRCM, we used different methods to detect code hallucinations.

**Results** The experimental results are shown in Table 4. Our experiments demonstrate that HalluCodeDetector outperforms baseline methods in hallucination detec-

**Table 4** The performance of different methods in code hallucination detection

| Method | HumanEval+ | | MBPP | | EvoCodeBench | |
|---|---|---|---|---|---|---|
| | AUPRC | AUROC | AUPRC | AUROC | AUPRC | AUROC |
| Random | 0.70 | 0.50 | 0.80 | 0.50 | 0.27 | 0.50 |
| OrdinaryLLM | 0.88 | 0.70 | 0.88 | 0.59 | 0.22 | 0.47 |
| LYNX | 0.89 | 0.71 | 0.88 | 0.60 | 0.25 | 0.46 |
| HalluCD-METEOR | 0.89 | 0.80 | 0.85 | 0.62 | 0.66 | 0.81 |
| HalluCD-BLEU | 0.89 | 0.78 | 0.87 | 0.62 | 0.63 | 0.79 |
| HalluCD-CodeBLEU | 0.91 | 0.82 | 0.88 | 0.63 | 0.70 | 0.85 |
| HalluCD-Prompt | 0.88 | 0.75 | **0.91** | **0.68** | 0.59 | 0.76 |
| HalluCD-MRCM | **0.93** | 0.87 | 0.89 | 0.64 | **0.74** | **0.88** |

tion across both HumanEval+ and MBPP datasets. Our MRCM-based method and Prompt-based method achieved SOTA results. The MRCM-based method achieved an AUPRC score of 0.93 and an AUROC score of 0.87 on HumanEval+, which is a 4.5% improvement on AUPRC and a 22.5% improvement on AUROC compared to LYNX. The Prompt-based method achieved an AUPRC score of 0.91 and an AUROC score of 0.68 on MBPP, which is a 3.4% improvement on AUPRC and a 13.3% improvement on AUROC compared to LYNX. The results demonstrate the effectiveness of static consensus verification.

Notably, HalluCodeDetector-MRCM maintains superior performance on EvoCodeBench–a repository-level dataset with complex multi-file dependencies, explicit library interactions, and a high hallucination rate (73.0%)–which better simulates real-world software development scenarios. Specifically:

**AUROC superiority** HalluCodeDetector-MRCM achieves an AUROC of 0.88 on EvoCodeBench, outperforming the SOTA baseline LYNX by 91.3% (LYNX: 0.46) and other static metric variants by 3.5%–11.4%. This indicates that MRCM's multi-dimensional consistency measurement effectively captures structural and logical patterns in complex code, whereas LYNX's semantic reasoning struggles with cross-file dependencies and intricate code interactions.

**AUPRC robustness** Despite EvoCodeBench's severe class imbalance (73.0% hallucinated samples), HalluCodeDetector-MRCM achieves an AUPRC of 0.74–far exceeding LYNX (0.25) and OrdinaryLLM (0.22). This validates MRCM's ability to prioritize stable discriminative features in imbalanced scenarios, avoiding the precision degradation that plagues text-only or LLM-based methods.

Among the different variants of HalluCodeDetector, we found the BLEU-based variant performed poorly, as it primarily measures surface-level textual similarity, failing to capture logical or semantic consistency.

CodeBLEU improves on BLEU by introducing structural features such as AST. However, its rigid weighting mechanism has problems in areas such as variable renaming and logic reconstruction, leading to misjudgments.

The HalluCodeDetector variant utilizing METEOR also exhibited suboptimal performance. Although METEOR incorporates synonym/stem matching and explicit recall mechanisms designed for natural language evaluation, these features translate poorly to the structural and functional nuances of code.

The Prompt-based variant achieved competitive results by leveraging GPT-4o's semantic understanding. However, due to the need to frequently communicate with the LLM, this method is extremely time-consuming.

Our proposed HalluCodeDetector-MRCM metric outperforms all other methods except HalluCodeDetector-Prompt by combining syntactic, logical, and textual features with adaptive weights. The variance-based dynamic adjustment of feature weights ensures a robust distinction between consistent and hallucinated outputs.

These results validate our hypothesis that semantic and functional consistency across multiple stochastic LLM responses serves as a reliable indicator of hallucination likelihood, even without code execution or external resources.

Another advantage of HalluCodeDetector lies in its ability to detect not only functional hallucinations (i.e., code that fails test cases, which defines our ground truth) but also non-functional hallucinations that escape test case validation–such as dead code hallucination. Dead code refers to redundant code segments that never execute during runtime (e.g., unreachable conditional branches, unused variable assignments, or redundant computations) but do not affect the pass/fail result of test cases, as they do not interfere with the core functionality verified by tests. For such non-functional hallucinations, traditional test-based validation typically provides no feedback, but HalluCodeDetector's consistency-based mechanism effectively captures them: the redundant code in hallucinated samples introduces extra structural nodes (in AST) and irrelevant logical edges (in DFG) that are absent in correct, concise samples. This leads to significantly lower similarity scores between dead code-containing samples and other consistent correct samples.

### 4.3 RQ2: Effectiveness of each feature of MRCM

**Approach** To verify the contribution of each feature of MRCM, various simplified variants of MRCM are tested on HumanEval+.

**Results** As shown in Table 5, the combination of AST and data flow (DF) similarity achieves the strongest performance among all dual-feature variants (AUPRC=0.84, AUROC=0.73), significantly outperforming text-based combinations such as Lev+Jaccard (AUPRC=0.76, AUROC=0.59). This is because AST captures the syntactic skeleton of the code, while DF analyzes the logical dependencies of the variable definition-use chain. The combination of the two can identify the deep functional consistency of the code and avoid the misleading of surface text similarity. For example, in the variable renaming or logic refactoring scenario in Fig. 3, the text feature fails, but AST+DF is still able to maintain a high degree of discrimination. This confirms that syntactic structure and logical dependencies are crucial for detecting functional consistency in code.

**Table 5** Performance of various simplified variants of MRCM

| Method | AUPRC | AUROC |
|---|---|---|
| AST | 0.79 | 0.63 |
| DF | 0.77 | 0.61 |
| Lev | 0.77 | 0.61 |
| Jaccard | 0.76 | 0.59 |
| AST + DF | 0.84 | 0.73 |
| AST + Lev | 0.80 | 0.64 |
| AST + Jaccard | 0.83 | 0.72 |
| DF + Lev | 0.78 | 0.61 |
| DF + Jaccard | 0.81 | 0.67 |
| Lev + Jaccard | 0.76 | 0.59 |
| AST + DF + Lev | 0.91 | 0.82 |
| AST + DF + Jaccard | 0.90 | 0.81 |
| AST + Lev + Jaccard | 0.89 | 0.79 |
| DF + Lev + Jaccard | 0.83 | 0.72 |
| MRCM | **0.93** | **0.87** |

While text-based features (Levenshtein ratio and Jaccard similarity) show limited effectiveness when used alone, they provide complementary information when combined with structural features. Adding Levenshtein to AST+DF improves AUPRC by 8.3% and AUROC by 12.3%.

The complete MRCM achieves optimal performance (AUPRC=0.93, AUROC=0.87), outperforming the best three-feature variant (AST+DF+Lev) by 2.2% in AUPRC and 6.1% in AUROC. This validates our dynamic weighting mechanism's ability to leverage complementary strengths: AST captures syntactic patterns, DF identifies logical dependencies, Levenshtein detects textual variations, and Jaccard measures lexical overlap.

### 4.4 RQ3: Efficiency of HalluCodeDetector

**Approach** In this experiment, we evaluate the time consumption of different methods for hallucination detection.

**Results** Since there is an order of magnitude difference in time cost between the static analysis-based methods (HalluCodeDetector-METEOR/BLEU/CodeBLEU/MRCM) and the LLM interaction-based methods (HalluCodeDetector-Prompt/OrdinaryLLM/LYNX), in order to avoid visualization distortion caused by scale differences, the experimental results are presented separately in Figs. 6 and 7.

As shown in Figs. 6 and 7, the average time cost of HalluCodeDetector-METEOR, HalluCodeDetector-BLEU and HalluCodeDetector-CodeBLEU for hallucination detection is 0.008 seconds, and the time cost of HalluCodeDetector-MRCM is 0.016 seconds, which is longer than the former three. This is because HalluCodeDetector-MRCM adds the step of adjusting feature weights. Although HalluCodeDetector-MRCM takes longer, AUPRC is improved by more than 1.1% and AUROC is improved by more than 4.1%.

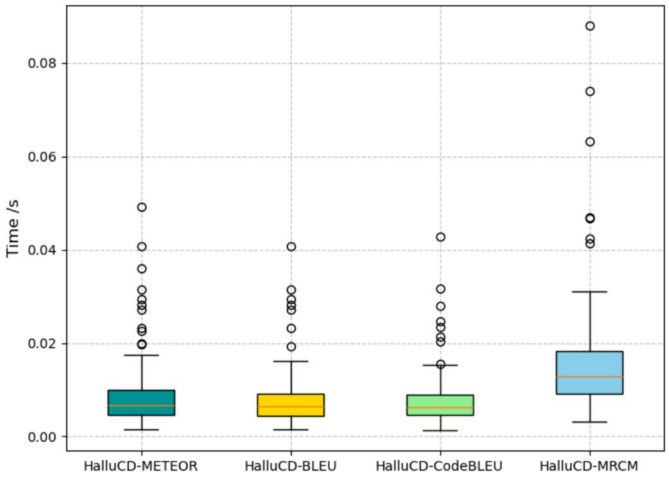



**Fig. 6** Time cost comparison between HalluCD-METEOR, HalluCD-BLEU, HalluCD-CodeBLEU and HalluCD-MRCM

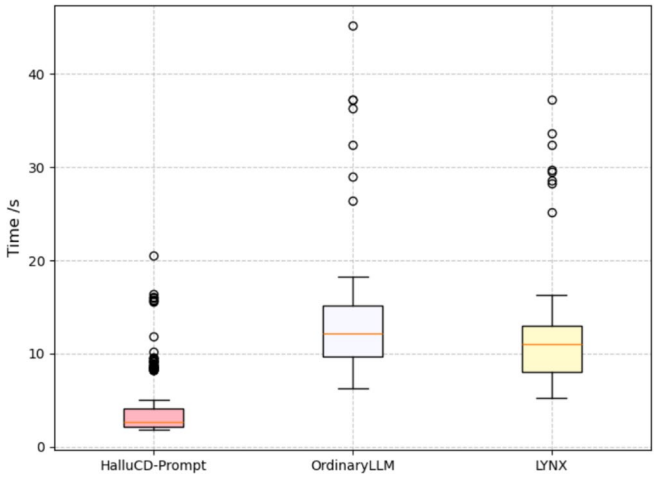



**Fig. 7** Time cost comparison between HalluCD-Prompt, OrdinaryLLM and LYNX

The time cost of HalluCodeDetector-Prompt, OrdinaryLLM, and LYNX is higher because they need to wait for the response of LLM. Among them, OrdinaryLLM and LYNX only need to obtain the answer of the LLM once, while HalluCodeDetector-Prompt needs to obtain the answer multiple times, but the question of HalluCodeDetector-Prompt is simpler, so it takes less time. At the same time, the AUPRC score and AUROC score of HalluCodeDetector-Prompt are improved by 1.1% and 9.1% respectively compared with LYNX.

### 4.5 RQ4: The choice of LLM for samples generation

**Approach** To evaluate the generalizability of HalluCodeDetector across different LLMs, we extend our experiments to code samples generated by diverse SOTA LLMs. Specifically, we select GPT-4o, Qwen3-235B-A22B (Yang et al. 2025) and Llama3.1-405B (Grattafiori et al. 2024) as additional code generation models, covering varying architectures and training paradigms. We then apply all HalluCodeDetector variants and baseline methods to evaluate the likelihood of hallucination in generated samples on HumanEval+.

**Results** HalluCodeDetector shows strong cross-model generalization ability. Table 6 shows that HalluCodeDetector-METEOR, HalluCodeDetector-BLEU and HalluCodeDetector-CodeBLEU perform close to LYNX in code hallucination detection tasks of different LLMs, while HalluCodeDetector-Prompt and HalluCodeDetector-MRCM perform better than LYNX.

HalluCodeDetector-MRCM leads LYNX by 4% $\sim$ 6% in AUPRC and 1% $\sim$ 18% in AUROC on different LLMs. This result verifies the robustness of the method, and the core reason is that the multi-dimensional features of MRCM can be generalized to the output styles of different LLMs. MRCM automatically focuses on features with high stability through variance-driven weight allocation, reducing model-specific interference.

### 4.6 RQ5: The impact of the number of samples

**Approach** While sample-based methods are expected to perform better with larger sample sizes, this comes at a higher computational cost. Therefore, we study the performance when the sample size varies.

**Results** From the data in Table 7, we can see that as the number of samples (N) increases from 2 to 10, the Spearman rank correlation coefficients of all HalluCodeDetector variants show an upward trend. Among them, the MRCM and Prompt methods can extract stable features from a small number of samples more quickly because they combine dynamic weight mechanisms and semantics, so these two methods reach near peak values when N=5. The METEOR/BLEU/CodBLEU method based on text similarity requires more samples to alleviate misjudgment, but is limited by the ability to express features, and the final performance is still poor.

When $N \geq 6$, the contribution of increasing samples to performance improvement is significantly reduced. The possible reason is that the diversity of samples generated by LLM can effectively cover potential correct patterns in the early stage ($N = 2 \sim 6$), while subsequent samples may introduce redundancy or noise.

**Table 6** Performance of different methods in code hallucination detection on different LLMs

| Method | HumanEval+(GPT-4o) | | HumanEval+(Qwen3) | | HumanEval+(Llama3.1) | |
|---|---|---|---|---|---|---|
| | AUPRC | AUROC | AUPRC | AUROC | AUPRC | AUROC |
| Random | 0.65 | 0.50 | 0.68 | 0.50 | 0.62 | 0.50 |
| OrdinaryLLM | 0.75 | 0.65 | 0.80 | 0.64 | 0.65 | 0.67 |
| LYNX | 0.78 | 0.67 | 0.82 | 0.65 | 0.66 | 0.69 |
| HalluCD-METEOR | 0.76 | 0.64 | 0.82 | 0.65 | 0.65 | 0.65 |
| HalluCD-BLEU | 0.77 | 0.64 | 0.82 | 0.64 | 0.65 | 0.66 |
| HalluCD-CodeBLEU | 0.77 | 0.64 | 0.83 | 0.67 | 0.66 | 0.66 |
| HalluCD-Prompt | 0.80 | 0.69 | 0.85 | 0.74 | 0.68 | 0.70 |
| HalluCD-MRCM | **0.81** | **0.72** | **0.87** | **0.77** | **0.69** | **0.70** |

**Table 7** The performance of HalluCodeDetector methods versus the number of samples (N)

| Method | Spearman's RankCC | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | N=2 | N=3 | N=4 | N=5 | N=6 | N=7 | N=8 | N=9 | N=10 |
| HalluCD-METEOR | 0.31 | 0.35 | 0.37 | 0.39 | 0.41 | 0.41 | 0.40 | 0.40 | 0.40 |
| HalluCD-BLEU | 0.28 | 0.34 | 0.35 | 0.38 | 0.41 | 0.40 | 0.40 | 0.40 | 0.40 |
| HalluCD-CodeBLEU | 0.32 | 0.38 | 0.42 | 0.44 | 0.47 | 0.46 | 0.45 | 0.45 | 0.45 |
| HalluCD-Prompt | 0.49 | 0.56 | 0.61 | 0.62 | 0.62 | 0.62 | 0.62 | 0.61 | 0.62 |
| HalluCD-MRCM | 0.46 | 0.54 | 0.59 | 0.60 | 0.60 | 0.60 | 0.59 | 0.59 | 0.60 |

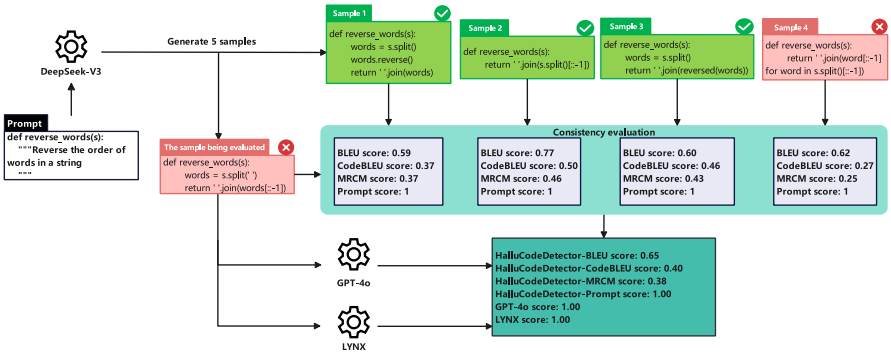



**Fig. 8** DeepSeek-V3 generated 3 correct answer samples and 2 incorrect answer samples for a question, while the HalluCodeDetector variant using the MRCM metric gave the most appropriate score for the evaluated sample

## 5 Case study

### 5.1 TP case

To concretely demonstrate the advantages of MRCM over traditional similarity metrics and the ability of HalluCodeDetector to detect hallucinations, we provide a detailed case study in Fig. 8.

To concretely demonstrate how MRCM differentiates correct from incorrect samples and its advantages over traditional metrics, we expand the string reversal case with detailed step-by-step calculations. As shown in Fig. 8, LLM (DeepSeek-V3) generates 5 solutions for reversing word order in a string (input "hello world" → expected output "world hello"): 3 correct samples and 2 incorrect samples. The reason for the error in the evaluated sample is that it ignores the case of consecutive spaces, and the reason for the error in sample 4 is that it reverse the entire string instead of the word order. We calculate the MRCM score of each sample by following the formulas in Section 2.2, and verify how MRCM captures consistency differences between correct/incorrect samples. The calculation steps are as follows:

**Table 8** The similarity of each feature between $S_0$ and four other samples

| Sample | AST-Sim | DF-Sim | Lev-Ratio | Jaccard-Sim |
|---|---|---|---|---|
| $S_1$ | 0.28 | 0.50 | 0.45 | 0.33 |
| $S_2$ | 0.39 | 0.33 | 0.74 | 0.93 |
| $S_3$ | 0.28 | 0.50 | 0.86 | 0.43 |
| $S_4$ | 0.14 | 0.22 | 0.49 | 0.82 |

Step 1: Calculate Four Core Feature Similarities. MRCM first quantifies similarity across four complementary dimensions, each targeting a distinct aspect of code consistency. For the evaluated sample $S_0$, we calculate its similarity with each of the other 4 samples using the formulas in Section 2.2: AST-Sim: Captures syntactic structural consistency after normalization. $S_0$ uses split(' ') while other samples use split()–this difference leads to divergent AST node structures. Thus, $S_0$'s AST-Sim with other samples ($S_1$: 0.28, $S_2$: 0.39, $S_3$: 0.28, $S_4$: 0.14) is significantly lower than if the syntactic structure were consistent. DF-Sim: Measures logical dependencies via data flow graph (DFG) edge overlap. $S_0$'s DFG includes an edge "split(' ') → join(' ')" that incorrectly enforces single-space separation, while correct samples have edges "split() → join(' ')" that handle arbitrary whitespace. Lev-Ratio: Quantifies textual edit distance (normalized to [0,1]). $S_0$ 's text is highly similar to $S_2$ (0.74) and $S_3$ (0.86). However, this surface-level similarity is misleading–MRCM does not over-rely on this feature, as shown in subsequent weighting steps. Jaccard-Sim: Evaluates lexical overlap of key identifiers. $S_0$ shares most identifiers (e.g., split, join, return) with all samples, leading to relatively high Jaccard-Sim with $S_2$ (0.93) and $S_4$ (0.82). This confirms that lexical overlap alone is insufficient to detect logical errors–a limitation MRCM addresses by combining with structural and data flow features.
Step 2: Next, MRCM computes the mean similarity and variance of each feature across $S_0$'s comparisons with ($S_1$–$S_4$), then assigns weights inversely proportional to variance. This "low-variance, high-weight" principle prioritizes features that are stable: Mean: For $S_0$, the mean AST-Sim is 0.27, mean DF-Sim is 0.39, mean Lev-Ratio is 0.64, and mean Jaccard-Sim is 0.63. Variance: AST-Sim has the lowest variance (0.007) because $S_0$'s structural inconsistency is consistent across all comparisons (no extreme fluctuations). Jaccard-Sim has the highest variance (0.064) because it is sensitive to minor lexical differences. Weights: As shown in Table 9, AST-Sim is assigned the highest weight (0.51) because its low variance indicates stable discriminative power (structural differences reliably separate correct and incorrect samples). DF-Sim is assigned the second-highest weight (0.29) for the same reason. Lev-Ratio (0.14) and Jaccard-Sim (0.06) receive low weights because their higher variance makes them less reliable for distinguishing logical correctness.

**Table 9** The mean, variance, and weights of each feature between $S_0$ and four other samples

| | AST-Sim | DF-Sim | Lev-Ratio | Jaccard-Sim |
|---|---|---|---|---|
| MEAN | 0.27 | 0.39 | 0.64 | 0.63 |
| VARIANCE | 0.007 | 0.014 | 0.029 | 0.064 |
| WEIGHT | 0.51 | 0.29 | 0.14 | 0.06 |

Step 3: Compute MRCM Scores for $S_0$. For each pair ($S_0$,$S_j$) (j=1–4), MRCM combines the four feature similarities using the dynamic weights. These scores reflect the weighted combination of structural, logical, textual, and lexical consistency. Even though $S_0$ has high textual/lexical similarity with some samples, the high weights assigned to AST-Sim and DF-Sim pull the overall MRCM scores downward–accurately reflecting $S_0$'s logical incorrectness.

### 5.2 Error analysis: FN case

The primary source of FN (hallucinated code missed by HalluCodeDetector) is "consistently wrong" generations–LLMs repeatedly generate the same erroneous code across all stochastic samples, leading to high consistency. We provide a detailed case study in Fig. 9.

The task requires generating a function to return the number of days in a specified year and month. The correct API for this task is calendar.monthrange(year, month), a standard library function that takes year as the first parameter and month as the second, returning a tuple (weekday_of_first_day, number_of_days). The number of days is extracted via the second element of the tuple.

The LLM misuses the API by reversing the parameter order (month first, year second) across all 4 stochastic samples. When the LLM repeatedly makes the same API misuse, the structural/logical consistency between samples masks the functional error. HalluCodeDetector's static consistency check cannot distinguish between "consistently correct" and "consistently wrong" code. To address this, our future work will integrate lightweight API knowledge bases to cross-validate consistent outputs.

### 5.3 Error analysis: FP case

FP (false positive) occurs when a code sample is functionally correct but structurally/logically inconsistent with other randomly generated codes, causing HalluCodeDetector to give a low consistency score and wrongly classify it as a hallucination. This scenario highlights a limitation of HalluCodeDetector's consistency-based logic: it is unable to distinguish between "rare but correct solutions" and "disagreements caused by hallucinations". We provide a detailed case study in Fig. 10.

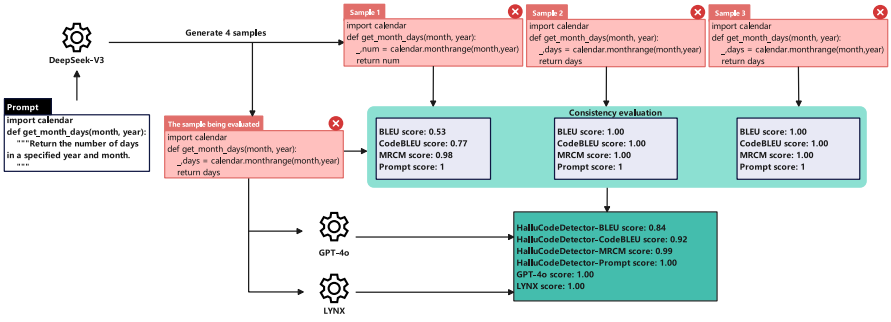



**Fig. 9** DeepSeek-V3 generated 4 incorrect answer samples for a question, all 4 samples share the exact same API misuse pattern

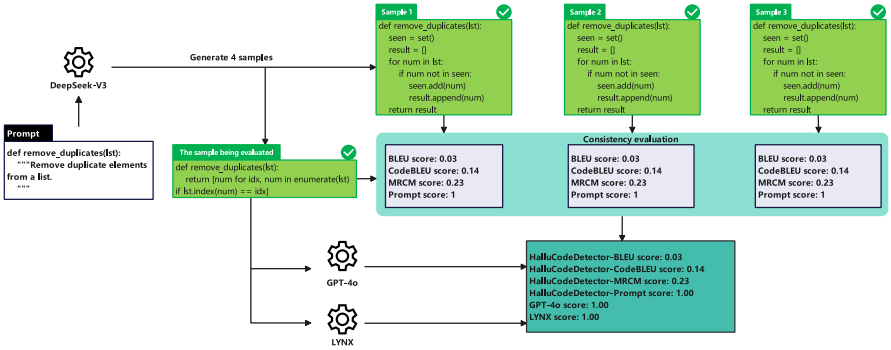



**Fig. 10** DeepSeek-V3 generated 4 correct answer samples for a question

The task requires generating a function to remove duplicate elements from a list while preserving the original order of the first occurrence. Valid solutions can adopt multiple algorithms, including: 1.HashSet-based approach: Use a set to track seen elements and a list to preserve order. 2.List comprehension + index check: Leverage list index lookup to retain the first occurrence, avoiding explicit set usage.

The core limitation exposed by this FP case lies in the inherent characteristic of static consistency metrics: they prioritize syntactic and logical structural alignment but lack the ability to directly verify functional equivalence. For the rare but correct list comprehension-based sample, its structural and logical divergence from the mainstream HashSet-based samples leads to low consistency scores, triggering misclassification. In contrast, the HalluCodeDetector-Prompt variant–relying on GPT-4o's semantic understanding–successfully identifies the functional consistency between the two algorithmic implementations, resulting in a full score (Prompt score=1.00). This stark contrast demonstrates the complementary nature of static metrics and LLM-based functional analysis: static metrics offer high efficiency and low computational overhead, while LLM-based methods excel at capturing functional equivalence across structural variations.

## 6 Threats to validity

- **Programming language limitations.** The method is only verified on Python code. The syntax and semantic differences of other languages (such as JavaScript and C++) may cause indicators such as MRCM to fail, such as the difference in logical dependency analysis between dynamically typed languages and statically typed languages.
- **Risk of inconsistent functional details.** If the LLM have a misunderstanding of the "specific functional requirements" of a task, it may lead to all randomly generated samples containing similar functional errors while maintaining a high degree of structural/logical consistency, which may cause misjudgment by the HalluCodeDetector.

- **Tool dependency.** Relies on the accuracy of AST parsers and data flow analysis toolchains. Differences in parsing tools across languages may affect generalization.
- **Dependence on Test-Case Labels.** The ground truth for hallucination labeling relies on test case execution, which may miss subtle logical errors that do not trigger test failures (e.g., inefficient algorithms, latent security vulnerabilities).

## 7 Related work

### 7.1 Hallucination detection in natural language

Hallucination detection has been extensively explored across multiple domains, with research primarily focusing on natural language processing tasks such as question answering (Cobbe et al. 2021; Lauscher and Glava 2025; Rapp et al. 2024; Sun et al. 2024), information retrieval (Kwiatkowski et al. 2019; Cao et al. 2020; Zhang et al. 2023), and image captioning (Zeng et al. 2024; Liu et al. 2024b). A core insight from natural language research that motivates our work is the use of multi-response consistency as a signal for hallucination. For example, SelfCheckGPT (Manakul et al. 2023) proposes a zero-resource method that leverages the consistency of multiple stochastic outputs from LLMs to detect hallucinations, demonstrating that factually correct responses tend to be semantically consistent across generations. Rapp et al. (2024) further validate this insight through simulation studies on QA chatbots, showing that consistency-based detection achieves high precision while noting that such methods in natural language rely heavily on textual similarity, which is insufficient for code scenarios.

Other natural language hallucination detection methods include semantic entropy-based approaches (Farquhar et al. 2024), which measure the uncertainty of generated content, and knowledge alignment frameworks (Zhang et al. 2023) that integrate external knowledge bases to verify factual accuracy. However, these methods are ill-suited for code hallucination detection: code correctness depends on syntactic structure, logical dependencies, and API usage rather than semantic entropy or general knowledge alignment. For instance, a code snippet may be semantically coherent but contain fatal logical errors or API misuse–issues that natural language-focused methods fail to capture.

### 7.2 Code hallucination

Research on code hallucination has evolved around three core directions: taxonomy and benchmarking, mitigation, and detection. Our work builds on and addresses limitations in these areas.

#### 7.2.1 Taxonomy and benchmarking

Early work on code hallucination focuses on defining and categorizing the phenomenon. Liu et al. (2024a) establish the first comprehensive taxonomy of code hallucinations, identifying five primary categories (e.g., API misuse, logical errors).

Sarkar's research (Sarkar 2025) extends this taxonomy with a more application-centric classification, dividing code hallucinations into four distinct types: API and library hallucinations, logic and algorithm hallucinations, dependency and configuration hallucinations, and security and safety hallucinations. Tian et al. (2025) propose CodeHaluEval, a large-scale benchmark with 8,883 samples, systematically evaluating 17 LLMs to reveal reliability gaps in AI code generation. These studies provide critical foundations for our work by clarifying the types of hallucinations to target (e.g., structural inconsistencies, API errors) and supplying standardized datasets (HumanEval+ and MBPP) for evaluation. However, they focus on characterization rather than developing generalizable detection methods.

#### 7.2.2 Detection

Existing code hallucination detection methods can be divided into dynamic execution-based, static pattern-based, and LLM reasoning-based approaches, all of which have inherent limitations that our HalluCodeDetector addresses through a static consistency verification paradigm.

Dynamic execution-based methods, such as EvalPlus (Liu et al. 2023b), enhance benchmark test suites with 80× more test cases to expose undetected errors in LLM-generated code. The similarity with our work is the common goal of identifying functional incorrectness (hallucinations). However, the key difference lies in the reliance on runtime environments: EvalPlus requires complex runtime environment configurations and cannot be deployed in resource-constrained scenarios. Our HalluCodeDetector adopts a pure static analysis approach that eliminates the need for code execution, solving the deployment complexity and compatibility issues of dynamic methods.

Static pattern-based methods, such as CodeMirage (Agarwal et al. 2024), attempt static analysis through benchmark-driven pattern matching. The similarity is the avoidance of runtime execution. The critical difference is the generalizability: CodeMirage focuses on dataset construction and relies on pre-defined error patterns, making it ineffective for unseen hallucination types or new programming tasks. In contrast, our HalluCodeDetector leverages multi-response consistency, which is a task-agnostic signal, enabling generalizable detection across diverse code generation scenarios without pre-defining error patterns. This solves the poor generalizability problem of static pattern-based methods.

Etf (Entity Tracing Framework) (Maharaj et al. 2025) is a representative static method tailored for hallucination detection in code summaries. It constructs entity mapping between code snippets and their summaries, detecting hallucinations by identifying mismatches between code entities (e.g., variables, functions) and summary descriptions. The core similarity between Etf and our work is the adoption of static analysis without runtime execution. However, the critical difference lies in the target scenario: Etf focuses on code summaries (natural language-text alignment task), where hallucinations manifest as entity mismatches between text and code. Our work targets code generation scenarios.

LLM-based baseline methods (e.g., LYNX (Ravi et al. 2024)) rely on fine-tuned or pre-trained LLMs to reason about code correctness. These methods share with

our work the advantage of not requiring manual pattern definition, but differ significantly in efficiency and generalization: LYNX requires fine-tuning on specific datasets, leading to poor cross-model generalization). In contrast, our HalluCodeDetector does not require model fine-tuning, achieves cross-model generalization (validated on GPT-4o, Qwen3, Llama3.1), and has extremely low time overhead (0.016 seconds per sample). This solves the efficiency and generalization limitations of LLM reasoning-based detection methods.

In summary, existing detection methods either rely on dynamic execution (leading to deployment issues), focus on pre-defined patterns (lacking generalizability), or depend on external resources/LLM reasoning (inefficient and poorly portable). HalluCodeDetector fills this gap by proposing a generalizable, efficient, and code-aware static detection framework based on multi-response consistency, addressing the core limitations of each existing detection paradigm.

### 7.2.3 Mitigation

Mitigation strategies for code hallucination often rely on dynamic execution or automated repair. Automated Program Repair (APR) techniques (Fan et al. 2023), such as Codex edit mode, can fix erroneous code post-generation. Spracklen et al. (2024) highlight package hallucinations as a software supply chain threat and propose execution-based mitigation pathways, but their reliance on runtime validation shares the same limitations as other dynamic methods. Sarkar's research (Sarkar 2025) complements these approaches with a multi-layered mitigation framework, including prompt engineering, human-in-the-loop workflows, and model refinement.

Compared with existing mitigation methods, the core positioning of the method in this paper is "pre-detection and screening". Existing mitigation methods solve the problem of "how to correct the hallucination", while the method in this paper solves the problem of "which samples need to be corrected". The combination of the two can form an efficient solution covering the entire process of "detection-mitigation", thus resolving the issue of cost redundancy in current hallucination mitigation research.

## 8 Conclusions and future works

In this paper, we propose HalluCodeDetector, a static analysis framework for detecting hallucinations in LLM-generated code via sampling-based consistency verification. Grounded in the core insight that correct code generation exhibits high consistency across syntactic, semantic, and functional dimensions while hallucinated outputs diverge, our approach introduces the MRCM metric to quantify hallucination likelihood through multi-dimensional feature fusion and dynamic weight allocation. This work pioneers a purely static detection paradigm that eliminates reliance on dynamic code execution, external knowledge bases, or model fine-tuning, addressing critical limitations of existing methods. HalluCodeDetector outperforms baseline methods in hallucination detection on three datasets. The experimental results confirmed our hypothesis that semantic consistency among multiple random LLM responses can serve as a reliable indicator of the possibility of hallucinations even without code

execution or external resources. HalluCodeDetector can be used to automatically review LLM-generated code, reducing vulnerabilities caused by hallucinations.

To address current limitations and expand the scope of application, we plan to pursue the following directions:

1. Multi-Language Extension: Adapt HalluCodeDetector to support Java, C++, and JavaScript by optimizing AST parsing and data flow analysis for language-specific syntax and semantic features (e.g., pointer operations in C++).
2. Detection-Repair Pipeline Integration: Combine HalluCodeDetector with automated program repair (APR) techniques to build an end-to-end "detection-repair" framework, enabling not only identification but also correction of hallucinated code snippets.

**Acknowledgments** This stusy is supported by National Key Laboratory of Data Space Technology and System

**Author Contributions** T.H., Y.H. and Z.R. wrote the main manuscript text, X.C., Y.L. and Z.Z. completed the experiments.

**Data Availability** To facilitate the research and application, our code and data are available at https://github.com/HalluCodeDetector/HalluCodeDetector.

## Declarations

**Competing interests** The authors declare no competing interests.

## References

Agarwal, V., Pei, Y., Alamir, S.: Codemirage: Hallucinations in code generated by large language models. arXiv preprint arXiv:2408.08333 (2024)

Austin, J., Odena, A., Nye, M.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021)

Bag, S., Kumar, S.: An efficient recommendation generation using relevant jaccard similarity. Inf. Sci. **483**, 53–64 (2019)

Banerjee, S., Lavie, A.: Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In: Proceedings of the Acl Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization, pp. 65–72 (2005)

Cao, S., Shi, J., Pan, L.: Kqa pro: A dataset with explicit compositional programs for complex question answering over knowledge base. arXiv preprint arXiv:2007.03875 (2020)

Chen, M.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)

Chen X, Hu X, Huang Y, et al.: Deep learning-based software engineering: progress, challenges, and opportunities. Science China Information Sciences68(1):111102 (2025)

Chen X, Xu F, Huang Y, et al.: Jit-smart: A multi-task learning framework for just-in-time defect prediction and localization. Proceedings of the ACM on Software Engineering 1(FSE):1–23 (2024)

Cobbe, K., Kosaraju, V., Bavarian, M.: Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 (2021)

Fan, Z., Gao, X., Mirchev, M.: Automated repair of programs from large language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp. 1469–1481 (2023)

Farquhar, S., Kossen, J., Kuhn, L.: Detecting hallucinations in large language models using semantic entropy. Nature **630**(8017), 625–630 (2024)

Grattafiori, A., Dubey, A., Jauhri, A.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
Huang Y, Chen Y, Chen X, et al.: Generative software engineering. arXiv preprint (2024) https://arxiv.org/abs/2403.02583
Huang Y, Hu X, Jia N, et al.: Learning code context information to predict comment locations. IEEE Transactions on Reliability 69(1):88–105 (2019)
Huang Y, Jia N, Chen X, et al.: Salient-class location: Help developers understand code change in code review. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 770–774 (2018)
Huang Y, Zhou Y, Chen X: Comment traps: How defective commentedout code augment defects in ai-assisted code generation. arXiv preprint (2025) https://arxiv.org/abs/2512.20334
Kwiatkowski, T., Palomaki, J., Redfield, O.: Natural questions: a benchmark for question answering research. Trans. Assoc. Computat. Linguist. **7**, 453–466 (2019)
Latif, A., Azam, F., Anwar, M.: Comparison of leading language parsers – antlr, javacc, sablecc, tree-sitter, yacc, bison. In: 2023 13th International Conference on Software Technology and Engineering (ICSTE), pp. 7–13 (2023). https://doi.org/10.1109/ICSTE61649.2023.00009
Lauscher, A., Glava, G.: How much do llms hallucinate across languages? on multilingual estimation of llm hallucination in the wild. arXiv preprint arXiv:2502.12769 (2025)
Li, J., Li, G., Zhang, X.: Evocodebench: An evolving code generation benchmark with domain-specific evaluations. Adv. Neural. Inf. Process. Syst. **37**, 57619–57641 (2024)
Liu, F., Liu, Y., Shi, L.: Exploring and evaluating hallucinations in llm-powered code generation. arXiv preprint arXiv:2404.00971 (2024a)
Liu, H., Xue, W., Chen, Y.: A survey on hallucination in large vision-language models. arXiv preprint arXiv:2402.00253 (2024b)
Liu, J., Xia, C.S., Wang, Y.: Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. Adv. Neural. Inf. Process. Syst. **36**, 21558–21572 (2023a)
Liu, J., Xia, C.S., Wang, Y.: Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. Adv. Neural. Inf. Process. Syst. **36**, 21558–21572 (2023b)
Maharaj, K., Munigala, V., Tamilselvam, S.: Etf: An entity tracing framework for hallucination detection in code summaries. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 30639–30652 (2025)
Manakul, P., Liusie, A., Gales, M.: Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In: Proceedings of the 2023 Conference On Empirical Methods in Natural Language Processing, pp. 9004–9017 (2023)
McDermott, M., Zhang, H., Hansen, L.: A closer look at auroc and auprc under class imbalance. Adv. Neural. Inf. Process. Syst. **37**, 44102–44163 (2024)
Nam, D., Macvean, A., Hellendoorn, V.: Using an llm to help with code understanding. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13 (2024)
Papineni, K., Roukos, S., Ward, T.: Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311–318 (2002)
Rapp, S., Maedche, A., Jainta, M.: Exploring the effects of consistency-based hallucination detection for llm-based qa chatbots: A simulation study. In: International Symposium on Chatbots and Human-Centered AI, Springer, pp. 33–47 (2024)
Ravi, S.S., Mielczarek, B., Kannappan, A.: Lynx: An open source hallucination evaluation model. arXiv preprint arXiv:2407.08488 (2024)
Ren, S., Guo, D., Lu, S.: Codebleu: a method for automatic evaluation of code synthesis. arXiv preprint arXiv:2009.10297 (2020)
Sarkar, S.: Auto-generated ai code hallucinations: Detection, impact, and mitigation strategies. Impact, and Mitigation Strategies (October 15, 2025) (2025)
Sedgwick, P.: Spearman's rank correlation coefficient. Bmj 349 (2014)
Spracklen, J., Wijewickrama, R., Sakib, A.: We have a package for you! a comprehensive analysis of package hallucinations by code generating llms. arXiv preprint arXiv:2406.10279 (2024)
Sun, Y., Yin, Z., Guo, Q.: Benchmarking hallucination in large language models based on unanswerable math word problem. arXiv preprint arXiv:2403.03558 (2024)
Tian, Y., Yan, W., Yang, Q.: Codehalu: Investigating code hallucinations in llms via execution-based verification. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 25300–25308 (2025)

Vaithilingam, P., Zhang, T., Glassman, E.L.: Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In: Chi Conference On Human Factors In Computing Systems Extended Abstracts, pp. 1–7 (2022)
Wang, J., Chen, Y.: A review on code generation with llms: Application and evaluation. In: 2023 IEEE International Conference on Medical Artificial Intelligence (MedAI), pp. 284–289 (2023). https://doi.org/10.1109/MedAI59581.2023.00044
Wen, W., Xue, X., Li, Y.: Code similarity detection using ast and textual information. Intern. J. Performabil. Eng. **15**(10), 2683 (2019)
Yang, A., Li, A., Yang, B.: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025)
Yujian, L., Bo, L.: A normalized levenshtein distance metric. IEEE Trans. Pattern Anal. Mach. Intell. **29**(6), 1091–1095 (2007)
Zeng, Z., Xie, Y., Zhang, H.: Meacap: Memory-augmented zero-shot image captioning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 14100–14110 (2024)
Zhang, S., Pan, L., Zhao, J.: The knowledge alignment problem: Bridging human and external knowledge for large language models. arXiv preprint arXiv:2305.13669 (2023)
Zhang, M., Press, O., Merrill, W., et al.: How language model hallucinations can snowball. In: Forty-first International Conference on Machine Learning (2024). https://openreview.net/forum?id=FPlaQyAGHu

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

## Authors and Affiliations

**Taicheng Huang[1] · Zhanhui Ren[1] · Yuan Huang[1] · Xiangping Chen[1] · Yi Liu[2] · Zibin Zheng[1]**

✉ Yuan Huang
huangyuan5@mail.sysu.edu.cn

✉ Yi Liu
liuyi14@pku.edu.cn

Taicheng Huang
chengcheng0711@outlook.com

Zhanhui Ren
renzhh9@mail2.sysu.edu.cn

Xiangping Chen
chenxp8@mail.sysu.edu.cn

Zibin Zheng
zhzibin@mail.sysu.edu.cn

[1] Sun Yat-sen University, Zhuhai 510275, China

[2] National Key Laboratory of Data Space Technology and System, Beijing 100091, China